# Assignment_03_part_03

October 19, 2026

## 0.1 EN3160 Assignment 3

### 0.1.1 A.H.T.M. Weerakoon

### 0.1.2 230689A

[3]:
```
%pip install --upgrade certifi
```

```
Requirement already satisfied: certifi in
c:\users\thiviru\appdata\local\programs\python\python310\lib\site-packages
(2026.6.17)Note: you may need to restart the kernel to use updated packages.


[notice] A new release of pip is available: 23.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip


Collecting certifi
  Downloading certifi-2026.10.18-py3-none-any.whl (134 kB)
     ---------------------------------------- 0.0/134.1 kB ? eta -:--:--
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     -------- ------------------------------ 30.7/134.1 kB 640.0 kB/s eta 0:00:01
     ------------------------ ----------- 92.2/134.1 kB 168.9 kB/s eta 0:00:01
     ------------------------ ----------- 92.2/134.1 kB 168.9 kB/s eta 0:00:01
     ------------------------ ----------- 92.2/134.1 kB 168.9 kB/s eta 0:00:01
     ------------------------ ----------- 92.2/134.1 kB 168.9 kB/s eta 0:00:01
     ------------------------ ----------- 92.2/134.1 kB 168.9 kB/s eta 0:00:01
     ------------------------------- --- 122.9/134.1 kB 150.2 kB/s eta 0:00:01
     ---------------------------------  133.1/134.1 kB 157.1 kB/s eta 0:00:01
     ---------------------------------- 134.1/134.1 kB 149.4 kB/s eta 0:00:00
Installing collected packages: certifi
  Attempting uninstall: certifi
    Found existing installation: certifi 2026.6.17
```

```
    Uninstalling certifi-2026.6.17:
      Successfully uninstalled certifi-2026.6.17
Successfully installed certifi-2026.10.18
```

### 0.1.3 Setup

[1]:
```
import ssl, certifi, functools
ssl._create_default_https_context = functools.partial(ssl.
 ↪create_default_context, cafile=certifi.where())
import os, time, copy, zipfile, urllib.request
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torchvision
import torchvision.transforms as transforms
from torchvision import datasets, models
from torchvision.models import ResNet18_Weights
%matplotlib inline

device = torch.device('cuda' if torch.cuda.is_available()
                      else 'mps' if torch.backends.mps.is_available() else␣
 ↪'cpu')
NUM_WORKERS = 0      # 0 is safest in Jupyter (especially on Windows); raise to␣
 ↪2-4 on Linux for speed
torch.manual_seed(0); np.random.seed(0)
print('Device:', device)
```

```
Device: cuda
```

### 0.1.4 Question 01

**Data loading**

[5]:
```
transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.
 ↪5, 0.5))])
batch_size = 50
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=NUM_WORKERS)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,␣
 ↪shuffle=False, num_workers=NUM_WORKERS)
```

```
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck')
```

```
100.0%
```

**Network parameters**

[6]:
```
Din = 3 * 32 * 32       # input size
H = 100                 # (a) hidden layer size
K = 10                  # classes

torch.manual_seed(0)
# Xavier-style initialisation: std 1e-5 (as in Listing 1) would make every␣
 ↪sigmoid output 0.5
# and the gradient reaching W1 almost zero.
w1 = (torch.randn(Din, H) * np.sqrt(2.0 / (Din + H))).to(device)
b1 = torch.zeros(H, device=device)
w2 = (torch.randn(H, K) * np.sqrt(2.0 / (H + K))).to(device)
b2 = torch.zeros(K, device=device)

# Hyperparameters
epochs = 10             # (c)
lr = 0.05
lr_decay = 0.95
reg = 1e-4

def forward(x):
    h = torch.sigmoid(x.mm(w1) + b1)        # hidden layer
    s = h.mm(w2) + b2                       # class scores (logits)
    return h, s

def accuracy(loader):
    correct = total = 0
    with torch.no_grad():
        for inputs, labels in loader:
            x = inputs.view(inputs.shape[0], -1).to(device)
            _, s = forward(x)
            correct += (s.argmax(1).cpu() == labels).sum().item()
            total += labels.size(0)
    return 100 * correct / total
```

**Training loop**

[7]:
```
loss_history, epoch_loss, epoch_train_acc, epoch_test_acc = [], [], [], []

for t in range(epochs):
    running_loss, correct, seen = 0.0, 0, 0
    for inputs, labels in trainloader:
        N = inputs.shape[0]
```

```
        x = inputs.view(N, -1).to(device)
        labels = labels.to(device)
        y = nn.functional.one_hot(labels, K).float()

        # Forward pass
        h, s = forward(x)
        log_p = s - torch.logsumexp(s, dim=1, keepdim=True)        #␣
↪numerically stable log-softmax
        p = log_p.exp()

        # (b) Cross-entropy loss + L2 regularisation
        loss = -(y * log_p).sum() / N + reg * (torch.sum(w1**2) + torch.
↪sum(w2**2))
        loss_history.append(loss.item())
        running_loss += loss.item()
        correct += (s.argmax(1) == labels).sum().item(); seen += N

        # Backpropagation
        ds = (p - y) / N                          # dL/dS
        dw2 = h.t().mm(ds) + 2 * reg * w2
        db2 = ds.sum(dim=0)
        dh = ds.mm(w2.t())
        dz1 = dh * h * (1 - h)                    # sigmoid derivative
        dw1 = x.t().mm(dz1) + 2 * reg * w1
        db1 = dz1.sum(dim=0)

        # Parameter update
        w1 -= lr * dw1; b1 -= lr * db1
        w2 -= lr * dw2; b2 -= lr * db2

    epoch_loss.append(running_loss / len(trainloader))
    epoch_train_acc.append(100 * correct / seen)
    epoch_test_acc.append(accuracy(testloader))
    print(f'Epoch {t+1}/{epochs}  loss {epoch_loss[-1]:.4f}  '
          f'train acc {epoch_train_acc[-1]:.2f}%  test acc {epoch_test_acc[-1]:.
↪2f}%')
    lr *= lr_decay
```

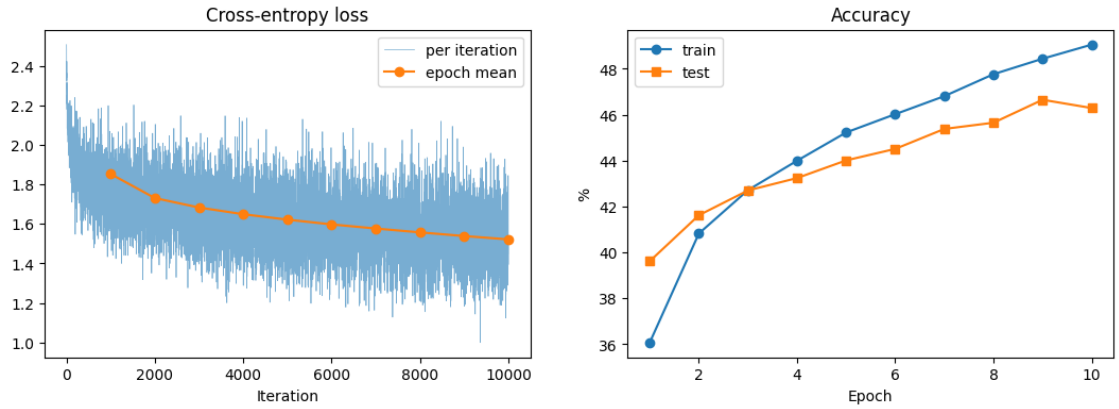
```
Epoch 1/10  loss 1.8534  train acc 36.08%  test acc 39.62%
Epoch 2/10  loss 1.7314  train acc 40.82%  test acc 41.62%
Epoch 3/10  loss 1.6834  train acc 42.71%  test acc 42.70%
Epoch 4/10  loss 1.6496  train acc 44.00%  test acc 43.24%
Epoch 5/10  loss 1.6220  train acc 45.23%  test acc 44.01%
Epoch 6/10  loss 1.5979  train acc 46.02%  test acc 44.51%
Epoch 7/10  loss 1.5769  train acc 46.81%  test acc 45.38%
Epoch 8/10  loss 1.5574  train acc 47.76%  test acc 45.65%
Epoch 9/10  loss 1.5389  train acc 48.44%  test acc 46.65%
```

```
Epoch 10/10  loss 1.5219  train acc 49.05%  test acc 46.29%
```

**Plots and final accuracies**

[8]:
```
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].plot(loss_history, lw=0.5, alpha=0.6, label='per iteration')
ax[0].plot(np.arange(1, epochs + 1) * len(trainloader), epoch_loss, 'o-',␣
 ↪label='epoch mean')
ax[0].set_title('Cross-entropy loss'); ax[0].set_xlabel('Iteration'); ax[0].
 ↪legend()
ax[1].plot(range(1, epochs + 1), epoch_train_acc, 'o-', label='train')
ax[1].plot(range(1, epochs + 1), epoch_test_acc, 's-', label='test')
ax[1].set_title('Accuracy'); ax[1].set_xlabel('Epoch'); ax[1].set_ylabel('%');␣
 ↪ax[1].legend()
plt.show()

train_acc = accuracy(trainloader)
test_acc = accuracy(testloader)
print(f'Training accuracy: {train_acc:.2f}%')
print(f'Test accuracy:     {test_acc:.2f}%')
```




```
Training accuracy: 50.18%
Test accuracy:     46.29%
```

### 0.1.5 Question 02

**Both models use the same conditions: Adam, learning rate 1e-3, batch size 64, cross-entropy loss, 10 epochs.**

**Data**

[3]:
```
mnist_tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.
 ↪1307,), (0.3081,))])
mnist_train = datasets.MNIST('./data', train=True, download=True,␣
 ↪transform=mnist_tf)
```

```
mnist_test = datasets.MNIST('./data', train=False, download=True,␣
 ↪transform=mnist_tf)
mnist_trainloader = torch.utils.data.DataLoader(mnist_train, batch_size=64,␣
 ↪shuffle=True, num_workers=NUM_WORKERS)
mnist_testloader = torch.utils.data.DataLoader(mnist_test, batch_size=1000,␣
 ↪shuffle=False, num_workers=NUM_WORKERS)
```

```
100.0%
100.0%
100.0%
100.0%
```

### (a) LeNet-5 and (b) small CNN

[4]:
```
# (a) LeNet-5 (LeCun et al., 1998): 28x28 input padded to 32x32 by padding=2
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Tanh(), nn.
 ↪AvgPool2d(2),    # 6 x 14 x 14
            nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(), nn.AvgPool2d(2))
 ↪      # 16 x 5 x 5
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(),
            nn.Linear(84, 10))
    def forward(self, x):
        return self.classifier(self.features(x))

# (b) Modern small CNN: 3 conv blocks with batch normalisation, ReLU, max␣
 ↪pooling, and dropout
def conv_block(cin, cout):
    return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1, bias=False),
                         nn.BatchNorm2d(cout), nn.ReLU(inplace=True), nn.
 ↪MaxPool2d(2))

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(conv_block(1, 32),      # 32 x 14 x 14
                                      conv_block(32, 64),     # 64 x 7 x 7
                                      conv_block(64, 128))    # 128 x 3 x 3
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Dropout(0.25),
            nn.Linear(128 * 3 * 3, 128), nn.ReLU(inplace=True), nn.Dropout(0.5),
            nn.Linear(128, 10))
```

```
    def forward(self, x):
        return self.classifier(self.features(x))

n_params = lambda m: sum(p.numel() for p in m.parameters())
print('LeNet-5 parameters:', n_params(LeNet5()))
print('SmallCNN parameters:', n_params(SmallCNN()))
```

```
LeNet-5 parameters: 61706
SmallCNN parameters: 241770
```

### Training and evaluation functions

[5]:
```
def run_epoch(model, loader, criterion, optimizer=None):
    training = optimizer is not None
    model.train(training)
    total_loss, correct, n = 0.0, 0, 0
    with torch.set_grad_enabled(training):
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            loss = criterion(out, y)
            if training:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            total_loss += loss.item() * x.size(0)
            correct += (out.argmax(1) == y).sum().item()
            n += x.size(0)
    return total_loss / n, 100 * correct / n

def train_mnist(model, epochs=10, lr=1e-3):
    torch.manual_seed(0)
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    hist = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}
    t0 = time.time()
    for ep in range(epochs):
        tr_loss, tr_acc = run_epoch(model, mnist_trainloader, criterion,␣
 ↪optimizer)
        te_loss, te_acc = run_epoch(model, mnist_testloader, criterion)
        for k, v in zip(hist, (tr_loss, tr_acc, te_loss, te_acc)):
            hist[k].append(v)
        print(f'Epoch {ep+1:2d}/{epochs}  train loss {tr_loss:.4f} acc {tr_acc:.
 ↪2f}%  |  '
              f'test loss {te_loss:.4f} acc {te_acc:.2f}%')
    hist['time'] = time.time() - t0
    return model, hist
```

### Train both models

[6]:
```
torch.manual_seed(0)
print('=== LeNet-5 ===')
lenet, hist_lenet = train_mnist(LeNet5())
torch.manual_seed(0)
print('\n=== SmallCNN ===')
cnn, hist_cnn = train_mnist(SmallCNN())
```

```
=== LeNet-5 ===
Epoch  1/10  train loss 0.2555 acc 92.62%  |  test loss 0.0877 acc 97.19%
Epoch  2/10  train loss 0.0727 acc 97.81%  |  test loss 0.0666 acc 97.95%
Epoch  3/10  train loss 0.0488 acc 98.53%  |  test loss 0.0655 acc 97.88%
Epoch  4/10  train loss 0.0398 acc 98.77%  |  test loss 0.0438 acc 98.52%
Epoch  5/10  train loss 0.0320 acc 99.03%  |  test loss 0.0517 acc 98.38%
Epoch  6/10  train loss 0.0269 acc 99.14%  |  test loss 0.0442 acc 98.61%
Epoch  7/10  train loss 0.0220 acc 99.27%  |  test loss 0.0415 acc 98.72%
Epoch  8/10  train loss 0.0209 acc 99.33%  |  test loss 0.0513 acc 98.57%
Epoch  9/10  train loss 0.0163 acc 99.47%  |  test loss 0.0421 acc 98.72%
Epoch 10/10  train loss 0.0144 acc 99.52%  |  test loss 0.0520 acc 98.68%

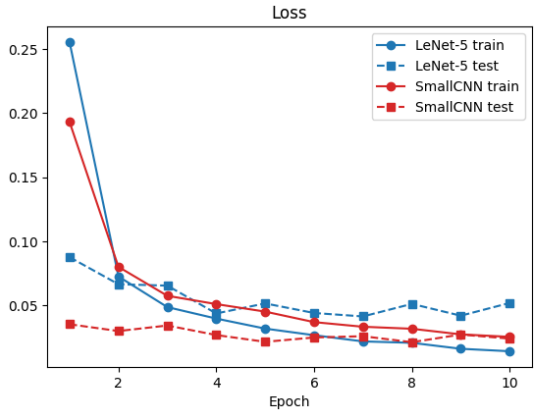
=== SmallCNN ===
Epoch  1/10  train loss 0.1932 acc 94.26%  |  test loss 0.0354 acc 98.87%
Epoch  2/10  train loss 0.0801 acc 97.69%  |  test loss 0.0301 acc 98.95%
Epoch  8/10  train loss 0.0319 acc 99.09%  |  test loss 0.0214 acc 99.32%
Epoch  9/10  train loss 0.0275 acc 99.14%  |  test loss 0.0274 acc 99.30%
Epoch 10/10  train loss 0.0256 acc 99.25%  |  test loss 0.0242 acc 99.32%
```

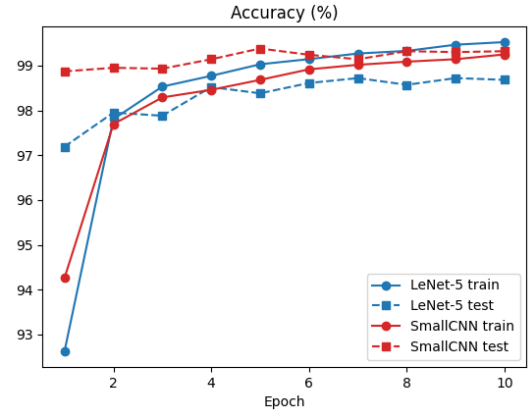
**(c) Per-epoch curves and final accuracies**

[7]:
```
ep = range(1, 11)
fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
for name, h, c in [('LeNet-5', hist_lenet, 'tab:blue'), ('SmallCNN', hist_cnn,␣
 ↪'tab:red')]:
    ax[0].plot(ep, h['train_loss'], 'o-', color=c, label=f'{name} train')
    ax[0].plot(ep, h['test_loss'], 's--', color=c, label=f'{name} test')
    ax[1].plot(ep, h['train_acc'], 'o-', color=c, label=f'{name} train')
    ax[1].plot(ep, h['test_acc'], 's--', color=c, label=f'{name} test')
ax[0].set_title('Loss'); ax[0].set_xlabel('Epoch'); ax[0].legend()
ax[1].set_title('Accuracy (%)'); ax[1].set_xlabel('Epoch'); ax[1].legend()
plt.show()

# Final accuracies, train set measured in eval mode (no dropout, BN running␣
 ↪stats)
crit = nn.CrossEntropyLoss()
print(f"{'Model':<10} {'Params':>8} {'Train acc':>10} {'Test acc':>9} {'Time␣
 ↪(s)':>9}")
for name, m, h in [('LeNet-5', lenet, hist_lenet), ('SmallCNN', cnn, hist_cnn)]:
    _, tr = run_epoch(m, mnist_trainloader, crit)
    _, te = run_epoch(m, mnist_testloader, crit)
```

```
    print(f'{name:<10} {n_params(m):>8} {tr:>9.2f}% {te:>8.2f}% {h["time"]:>9.
↪1f}')
```




| Model | Params | Train acc | Test acc | Time (s) |
|---|---|---|---|---|
| LeNet-5 | 61706 | 99.42% | 98.68% | 896.4 |
| SmallCNN | 241770 | 99.80% | 99.32% | 1039.9 |

### (c) Confusion matrices

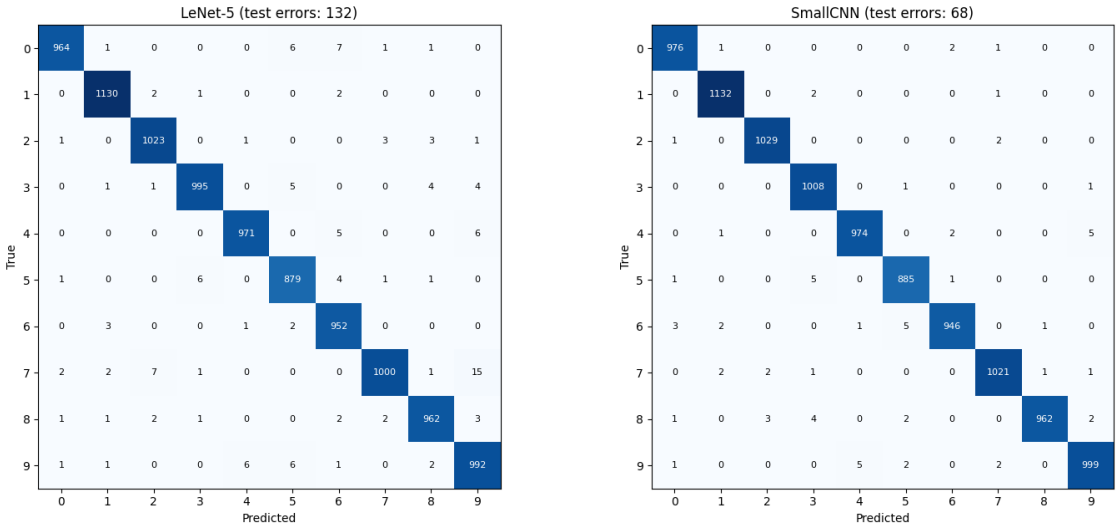
[8]:
```
def confusion_matrix(model, loader, K=10):
    cm = torch.zeros(K, K, dtype=torch.long)
    model.eval()
    with torch.no_grad():
        for x, y in loader:
            pred = model(x.to(device)).argmax(1).cpu()
            cm += torch.bincount(y * K + pred, minlength=K * K).view(K, K)
    return cm.numpy()

def plot_cm(cm, title, ax):
    ax.imshow(cm, cmap='Blues')
    ax.set_title(title); ax.set_xlabel('Predicted'); ax.set_ylabel('True')
    ax.set_xticks(range(10)); ax.set_yticks(range(10))
    thr = cm.max() / 2
    for i in range(10):
        for j in range(10):
            ax.text(j, i, cm[i, j], ha='center', va='center', fontsize=8,
                    color='white' if cm[i, j] > thr else 'black')

fig, ax = plt.subplots(1, 2, figsize=(15, 6.5))
for a, (name, m) in zip(ax, [('LeNet-5', lenet), ('SmallCNN', cnn)]):
    cm = confusion_matrix(m, mnist_testloader)
    plot_cm(cm, f'{name} (test errors: {cm.sum() - np.trace(cm)})', a)
    off = cm.copy(); np.fill_diagonal(off, 0)
```

```
    top = np.dstack(np.unravel_index(np.argsort(-off.ravel())[:3], off.
 ↪shape))[0]
    print(name, 'most frequent confusions (true -> predicted, count):',
          [(int(i), int(j), int(off[i, j])) for i, j in top])
plt.tight_layout(); plt.show()
```

```
LeNet-5 most frequent confusions (true -> predicted, count): [(7, 9, 15), (0, 6,
7), (7, 2, 7)]
SmallCNN most frequent confusions (true -> predicted, count): [(4, 9, 5), (5, 3,
5), (9, 4, 5)]
```




### 0.1.6 Question 03

**ResNet-18 pre-trained on ImageNet-1K classifies ants vs bees, once by fine-tuning all layers (a) and once as a frozen feature extractor (b). Both use SGD (lr 0.001, momentum 0.9), lr $\times 0.1$ every 7 epochs, 25 epochs.**

**Data**

[3]:
```
os.makedirs('data', exist_ok=True)
data_dir = 'data/hymenoptera_data'
if not os.path.isdir(data_dir):
    url = 'https://download.pytorch.org/tutorial/hymenoptera_data.zip'
    urllib.request.urlretrieve(url, 'data/hymenoptera_data.zip')
    with zipfile.ZipFile('data/hymenoptera_data.zip') as z:
        z.extractall('data')

mean, std = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]      # ImageNet␣
 ↪statistics
data_transforms = {
```

```
    'train': transforms.Compose([transforms.RandomResizedCrop(224), transforms.
 ↪RandomHorizontalFlip(),
                                 transforms.ToTensor(), transforms.
 ↪Normalize(mean, std)]),
    'val': transforms.Compose([transforms.Resize(256), transforms.
 ↪CenterCrop(224),
                               transforms.ToTensor(), transforms.
 ↪Normalize(mean, std)]),
}
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),␣
 ↪data_transforms[x]) for x in ['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=4,␣
 ↪shuffle=(x == 'train'),
                                              num_workers=NUM_WORKERS) for x in␣
 ↪['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes
print(dataset_sizes, class_names)
```

```
{'train': 244, 'val': 153} ['ants', 'bees']
```

**Training function**

[4]:
```
def train_tl(model, criterion, optimizer, scheduler, num_epochs=25):
    t0 = time.time()
    best_wts, best_acc = copy.deepcopy(model.state_dict()), 0.0
    hist = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
    for epoch in range(num_epochs):
        for phase in ['train', 'val']:
            model.train(phase == 'train')
            running_loss, running_corrects = 0.0, 0
            for inputs, labels in dataloaders[phase]:
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    if phase == 'train':
                        loss.backward(); optimizer.step()
                running_loss += loss.item() * inputs.size(0)
                running_corrects += (outputs.argmax(1) == labels).sum().item()
            if phase == 'train':
                scheduler.step()
            ep_loss = running_loss / dataset_sizes[phase]
            ep_acc = 100 * running_corrects / dataset_sizes[phase]
            hist[f'{phase}_loss'].append(ep_loss); hist[f'{phase}_acc'].
 ↪append(ep_acc)
            if phase == 'val' and ep_acc > best_acc:
```

```
            best_acc, best_wts = ep_acc, copy.deepcopy(model.state_dict())
        print(f"Epoch {epoch+1:2d}/{num_epochs}  train loss␣
↪{hist['train_loss'][-1]:.4f} "
              f"acc {hist['train_acc'][-1]:.2f}%  |  val loss␣
↪{hist['val_loss'][-1]:.4f} acc {hist['val_acc'][-1]:.2f}%")
    hist['time'], hist['best_acc'] = time.time() - t0, best_acc
    print(f"Done in {hist['time']/60:.1f} min, best val acc {best_acc:.2f}%")
    model.load_state_dict(best_wts)
    return model, hist
```

**(a) Fine-tuning**

[5]:
```
model_ft = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model_ft.fc = nn.Linear(model_ft.fc.in_features, len(class_names))
model_ft = model_ft.to(device)

criterion = nn.CrossEntropyLoss()
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
scheduler_ft = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
trainable = lambda m: sum(p.numel() for p in m.parameters() if p.requires_grad)
print('Trainable parameters:', trainable(model_ft))

model_ft, hist_ft = train_tl(model_ft, criterion, optimizer_ft, scheduler_ft,␣
↪num_epochs=25)
```

```
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to
C:\Users\THIVIRU/.cache\torch\hub\checkpoints\resnet18-f37072fd.pth

100.0%

Trainable parameters: 11177538
Epoch  1/25  train loss 0.5849 acc 68.85%  |  val loss 0.2073 acc 92.81%
Epoch  2/25  train loss 0.5414 acc 73.36%  |  val loss 0.2559 acc 89.54%
Epoch  3/25  train loss 0.6831 acc 72.95%  |  val loss 0.2418 acc 90.20%
Epoch  4/25  train loss 0.4057 acc 85.25%  |  val loss 0.3549 acc 84.31%
Epoch  5/25  train loss 0.4570 acc 81.15%  |  val loss 0.2329 acc 88.89%
Epoch  6/25  train loss 0.3460 acc 88.11%  |  val loss 0.2778 acc 90.85%
Epoch  7/25  train loss 0.3961 acc 84.84%  |  val loss 0.2354 acc 88.24%
Epoch  8/25  train loss 0.3260 acc 86.07%  |  val loss 0.1800 acc 92.81%
Epoch  9/25  train loss 0.4124 acc 82.38%  |  val loss 0.2111 acc 92.81%
Epoch 10/25  train loss 0.2773 acc 88.11%  |  val loss 0.1867 acc 93.46%
Epoch 11/25  train loss 0.3650 acc 83.20%  |  val loss 0.2012 acc 91.50%
Epoch 12/25  train loss 0.2597 acc 89.75%  |  val loss 0.1961 acc 90.85%
Epoch 13/25  train loss 0.3299 acc 85.25%  |  val loss 0.1966 acc 92.16%
Epoch 14/25  train loss 0.2799 acc 87.70%  |  val loss 0.2303 acc 90.85%
Epoch 15/25  train loss 0.2751 acc 88.52%  |  val loss 0.3138 acc 89.54%
Epoch 16/25  train loss 0.2374 acc 88.93%  |  val loss 0.2024 acc 90.85%
Epoch 17/25  train loss 0.2387 acc 88.11%  |  val loss 0.2360 acc 90.20%
Epoch 18/25  train loss 0.3252 acc 85.66%  |  val loss 0.2480 acc 91.50%
```

```
Epoch 19/25  train loss 0.1786 acc 92.62%  |  val loss 0.2105 acc 91.50%
Epoch 20/25  train loss 0.2637 acc 89.34%  |  val loss 0.1995 acc 92.16%
Epoch 21/25  train loss 0.2966 acc 87.30%  |  val loss 0.2027 acc 92.81%
Epoch 22/25  train loss 0.2504 acc 89.75%  |  val loss 0.1899 acc 93.46%
Epoch 23/25  train loss 0.1970 acc 91.80%  |  val loss 0.1937 acc 92.16%
Epoch 24/25  train loss 0.3163 acc 85.25%  |  val loss 0.1943 acc 91.50%
Epoch 25/25  train loss 0.2293 acc 91.39%  |  val loss 0.1905 acc 91.50%
Done in 2.1 min, best val acc 93.46%
```

**(b) Feature extractor**

[6]:
```
model_conv = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
for p in model_conv.parameters():
    p.requires_grad = False
model_conv.fc = nn.Linear(model_conv.fc.in_features, len(class_names))   # new␣
 ↪layer is trainable by default
model_conv = model_conv.to(device)

optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)
scheduler_conv = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
print('Trainable parameters:', trainable(model_conv))

model_conv, hist_conv = train_tl(model_conv, criterion, optimizer_conv,␣
 ↪scheduler_conv, num_epochs=25)
```

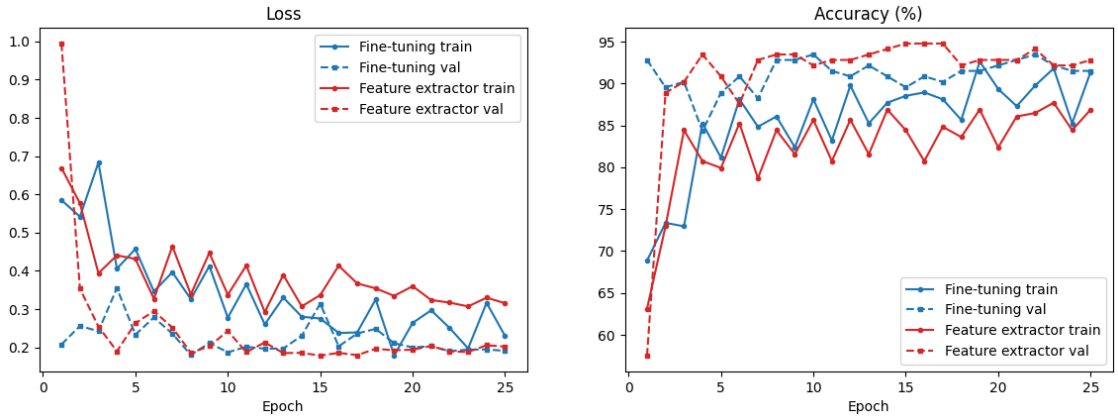
```
Trainable parameters: 1026
Epoch  1/25  train loss 0.6689 acc 63.11%  |  val loss 0.9944 acc 57.52%
Epoch  2/25  train loss 0.5765 acc 72.95%  |  val loss 0.3532 acc 88.89%
Epoch  3/25  train loss 0.3936 acc 84.43%  |  val loss 0.2540 acc 90.20%
Epoch  4/25  train loss 0.4403 acc 80.74%  |  val loss 0.1894 acc 93.46%
Epoch  5/25  train loss 0.4310 acc 79.92%  |  val loss 0.2631 acc 90.85%
Epoch  6/25  train loss 0.3270 acc 85.25%  |  val loss 0.2944 acc 87.58%
Epoch  7/25  train loss 0.4640 acc 78.69%  |  val loss 0.2514 acc 92.81%
Epoch  8/25  train loss 0.3381 acc 84.43%  |  val loss 0.1855 acc 93.46%
Epoch  9/25  train loss 0.4469 acc 81.56%  |  val loss 0.2027 acc 93.46%
Epoch 10/25  train loss 0.3372 acc 85.66%  |  val loss 0.2424 acc 92.16%
Epoch 11/25  train loss 0.4138 acc 80.74%  |  val loss 0.1871 acc 92.81%
Epoch 12/25  train loss 0.2920 acc 85.66%  |  val loss 0.2126 acc 92.81%
Epoch 13/25  train loss 0.3892 acc 81.56%  |  val loss 0.1848 acc 93.46%
Epoch 14/25  train loss 0.3069 acc 86.89%  |  val loss 0.1853 acc 94.12%
Epoch 15/25  train loss 0.3365 acc 84.43%  |  val loss 0.1781 acc 94.77%
Epoch 16/25  train loss 0.4134 acc 80.74%  |  val loss 0.1853 acc 94.77%
Epoch 17/25  train loss 0.3668 acc 84.84%  |  val loss 0.1793 acc 94.77%
Epoch 18/25  train loss 0.3540 acc 83.61%  |  val loss 0.1961 acc 92.16%
Epoch 19/25  train loss 0.3343 acc 86.89%  |  val loss 0.1919 acc 92.81%
Epoch 20/25  train loss 0.3596 acc 82.38%  |  val loss 0.1937 acc 92.81%
Epoch 21/25  train loss 0.3235 acc 86.07%  |  val loss 0.2030 acc 92.81%
Epoch 22/25  train loss 0.3167 acc 86.48%  |  val loss 0.1892 acc 94.12%
Epoch 23/25  train loss 0.3068 acc 87.70%  |  val loss 0.1876 acc 92.16%
```

```
Epoch 24/25  train loss 0.3296 acc 84.43%  |  val loss 0.2053 acc 92.16%
Epoch 25/25  train loss 0.3151 acc 86.89%  |  val loss 0.2016 acc 92.81%
Done in 1.9 min, best val acc 94.77%
```

### Comparison

[7]:
```
ep = range(1, 26)
fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
for name, h, c in [('Fine-tuning', hist_ft, 'tab:blue'), ('Feature extractor',␣
 ↪hist_conv, 'tab:red')]:
    ax[0].plot(ep, h['train_loss'], 'o-', ms=3, color=c, label=f'{name} train')
    ax[0].plot(ep, h['val_loss'], 's--', ms=3, color=c, label=f'{name} val')
    ax[1].plot(ep, h['train_acc'], 'o-', ms=3, color=c, label=f'{name} train')
    ax[1].plot(ep, h['val_acc'], 's--', ms=3, color=c, label=f'{name} val')
ax[0].set_title('Loss'); ax[0].set_xlabel('Epoch'); ax[0].legend()
ax[1].set_title('Accuracy (%)'); ax[1].set_xlabel('Epoch'); ax[1].legend()
plt.show()

print(f"{'Method':<18} {'Trainable params':>16} {'Best val acc':>13} {'Time␣
 ↪(min)':>11}")
for name, m, h in [('Fine-tuning', model_ft, hist_ft), ('Feature extractor',␣
 ↪model_conv, hist_conv)]:
    print(f"{name:<18} {trainable(m):>16,} {h['best_acc']:>12.2f}% {h['time']/
 ↪60:>11.1f}")
```



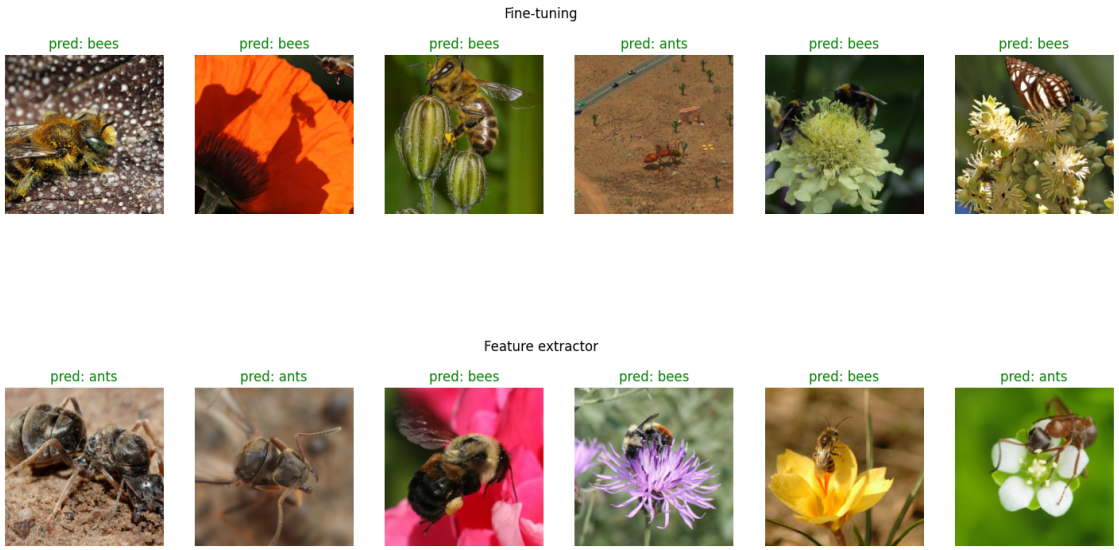

| Method | Trainable params | Best val acc | Time (min) |
|---|---|---|---|
| Fine-tuning | 11,177,538 | 93.46% | 2.1 |
| Feature extractor | 1,026 | 94.77% | 1.9 |

### Sample predictions

[8]:
```
def show_predictions(model, title, n=6):
    model.eval()
```

```
    inputs, labels = next(iter(torch.utils.data.
 ↪DataLoader(image_datasets['val'], batch_size=n, shuffle=True)))
    with torch.no_grad():
        preds = model(inputs.to(device)).argmax(1).cpu()
    fig, ax = plt.subplots(1, n, figsize=(3 * n, 3.3))
    for i in range(n):
        img = inputs[i].numpy().transpose(1, 2, 0) * np.array(std) + np.
 ↪array(mean)
        ax[i].imshow(np.clip(img, 0, 1)); ax[i].axis('off')
        ok = preds[i] == labels[i]
        ax[i].set_title(f'pred: {class_names[preds[i]]}', color='green' if ok␣
 ↪else 'red')
    fig.suptitle(title); plt.show()

show_predictions(model_ft, 'Fine-tuning')
show_predictions(model_conv, 'Feature extractor')
```




[ ]: